



IFAC – INSTITUTO FEDERAL DO ACRE

CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA
INTERNET

Locadora de Veículos

Documentação do Projeto Final – Programação Orientada a
Objetos

Integrantes:

João Gabriel Santos

Daniel Santos

Nathan Santos

Pedro Lucas

Jesumira Felix

Programação Orientada a Objetos

Professor(a): Jonas Pontes

06/07/2026

Sumário

1	Descrição geral do problema e da solução	2
1.1	O problema	2
1.2	A solução	2
2	Requisitos funcionais	3
3	Casos de uso	4
3.1	UC01 — Cadastrar cliente	4
3.2	UC02 — Cadastrar veículo na frota	4
3.3	UC03 — Criar locação	5
3.4	UC04 — Finalizar locação (devolução do veículo)	5
3.5	UC05 — Cancelar locação	6
4	Diagrama de classes	7
4.1	Classes de modelo e seus atributos/métodos principais	7
4.2	Relacionamentos principais	9
5	Boas práticas de POO aplicadas	10

1 Descrição geral do problema e da solução

1.1 O problema

Pequenas e médias locadoras de veículos frequentemente não têm acesso a sistemas de gestão robustos e ainda controlam sua frota, clientes e contratos de forma manual (papel, planilhas soltas ou até WhatsApp). Isso gera uma série de problemas:

- Dificuldade de saber, em tempo real, quais veículos estão disponíveis;
- Falta de controle sobre datas de devolução e possíveis atrasos;
- Ausência de histórico organizado de locações por cliente;
- Cálculo manual e sujeito a erro do valor a cobrar (diária, adicionais, multas).

1.2 A solução

O sistema **Locadora de Veículos** centraliza o fluxo completo de uma locação: cadastro de clientes (com CNH), funcionários, veículos (carros, motos e vans, cada categoria com sua própria regra de precificação) e locações. O sistema permite criar uma locação vinculando cliente, veículo, funcionário responsável, período e forma de pagamento, acompanhar seu status (reservada, em andamento, finalizada ou cancelada) e calcular automaticamente o valor total devido — incluindo multa por atraso na devolução.

O projeto foi desenvolvido como protótipo funcional em Java, com persistência simulada em memória (sem necessidade de banco de dados), e interface textual via terminal.

2 Requisitos funcionais

A tabela 1 apresenta os requisitos funcionais do sistema.

#	Requisito
RF01	O sistema deve permitir cadastrar, listar e consultar clientes, incluindo nome, e-mail, telefone e CNH.
RF02	O sistema deve permitir cadastrar veículos em diferentes categorias (Carro, Moto, Van), cada uma com sua própria regra de cálculo de diária.
RF03	O sistema deve permitir cadastrar funcionários responsáveis por formalizar as locações.
RF04	O sistema deve permitir criar uma locação vinculando um cliente, um veículo disponível, um funcionário, um período (data de início e devolução prevista), itens adicionais (opcional) e uma forma de pagamento (cartão, Pix ou dinheiro).
RF05	O sistema deve controlar a disponibilidade do veículo, marcando-o como indisponível durante a locação e disponível novamente ao ser finalizada ou cancelada.
RF06	O sistema deve calcular o valor total da locação ao finalizá-la, aplicando multa proporcional caso a devolução ocorra após a data prevista.

Tabela 1: Requisitos funcionais do sistema

3 Casos de uso

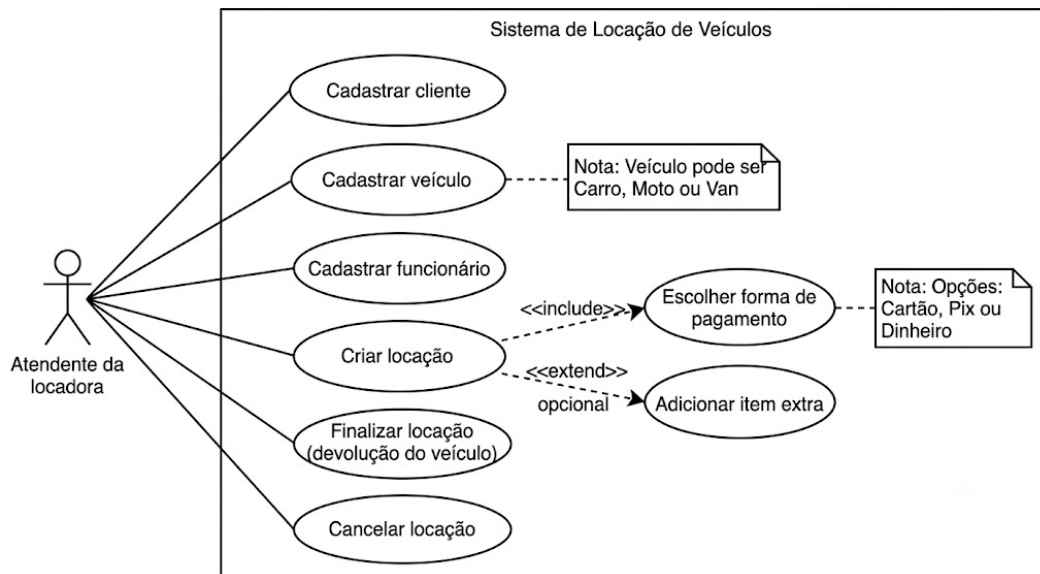


Figura 1: Diagrama de casos de uso do sistema Locadora de Veículos

3.1 UC01 — Cadastrar cliente

Ator principal: Atendente da locadora

Fluxo principal:

1. O atendente seleciona a opção "Cadastrar cliente".
2. O sistema solicita nome, e-mail, telefone e CNH.
3. O atendente informa os dados.
4. O sistema valida se o e-mail e a CNH já não estão cadastrados.
5. O sistema cria o cliente e exibe o ID gerado.

Fluxo alternativo: Se o e-mail ou a CNH já existirem, o sistema exibe mensagem de erro e retorna ao menu de clientes.

3.2 UC02 — Cadastrar veículo na frota

Ator principal: Atendente da locadora

Fluxo principal:

1. O atendente seleciona a categoria do veículo (Carro, Moto ou Van).
2. O sistema solicita placa, modelo, marca, ano, preço da diária base e os atributos específicos da categoria (nº de portas/ar-condicionado para carro, cilindrada para moto, capacidade de passageiros para van).
3. O sistema valida os dados informados.
4. O sistema cadastra o veículo com status "disponível" e exibe o ID gerado.

Fluxo alternativo: Se a placa, modelo/marca não forem informados ou o preço da diária for inválido, o sistema exibe mensagem de erro.

3.3 UC03 — Criar locação

Ator principal: Atendente da locadora (em nome do cliente)

Pré-condição: Cliente, veículo e funcionário já cadastrados; veículo disponível.

Fluxo principal:

1. O atendente seleciona "Criar nova locação".
2. O sistema solicita o ID do cliente, do veículo e do funcionário responsável.
3. O sistema solicita a data de início e a data prevista de devolução.
4. O sistema solicita a forma de pagamento (cartão, Pix ou dinheiro) e os dados específicos dela.
5. Opcionalmente, o atendente adiciona itens extras (seguro, GPS, cadeira infantil), informando nome e valor diário.
6. O sistema calcula o valor total previsto e marca o veículo como indisponível.
7. O sistema exibe o resumo da locação criada, com status inicial "Reservada".

Fluxo alternativo: Se o veículo não estiver disponível, ou a data de devolução prevista não for posterior à data de início, o sistema exibe mensagem de erro e cancela a operação.

3.4 UC04 — Finalizar locação (devolução do veículo)

Ator principal: Atendente da locadora

Pré-condição: Locação existente, ainda não finalizada nem cancelada.

Fluxo principal:

1. O atendente seleciona "Finalizar locação".
2. O sistema solicita o ID da locação e a data real de devolução.
3. O sistema calcula o valor total, incluindo multa proporcional caso a devolução seja posterior à data prevista.
4. O sistema marca o veículo como disponível novamente e o status da locação como "Finalizada".
5. O sistema processa o pagamento do valor total e exibe a confirmação.

Fluxo alternativo: Se a locação não existir ou já tiver sido finalizada/cancelada, o sistema exibe mensagem de erro.

3.5 UC05 — Cancelar locação

Ator principal: Atendente da locadora

Pré-condição: Locação existente e ainda não finalizada.

Fluxo principal:

1. O atendente seleciona "Cancelar locação".
2. O sistema solicita o ID da locação.
3. O sistema verifica se a locação ainda não foi finalizada.
4. O sistema altera o status da locação para "Cancelada" e libera o veículo (disponível novamente).

Fluxo alternativo: Se a locação já estiver com status "Finalizada", o sistema impede o cancelamento e exibe mensagem de erro.

4 Diagrama de classes

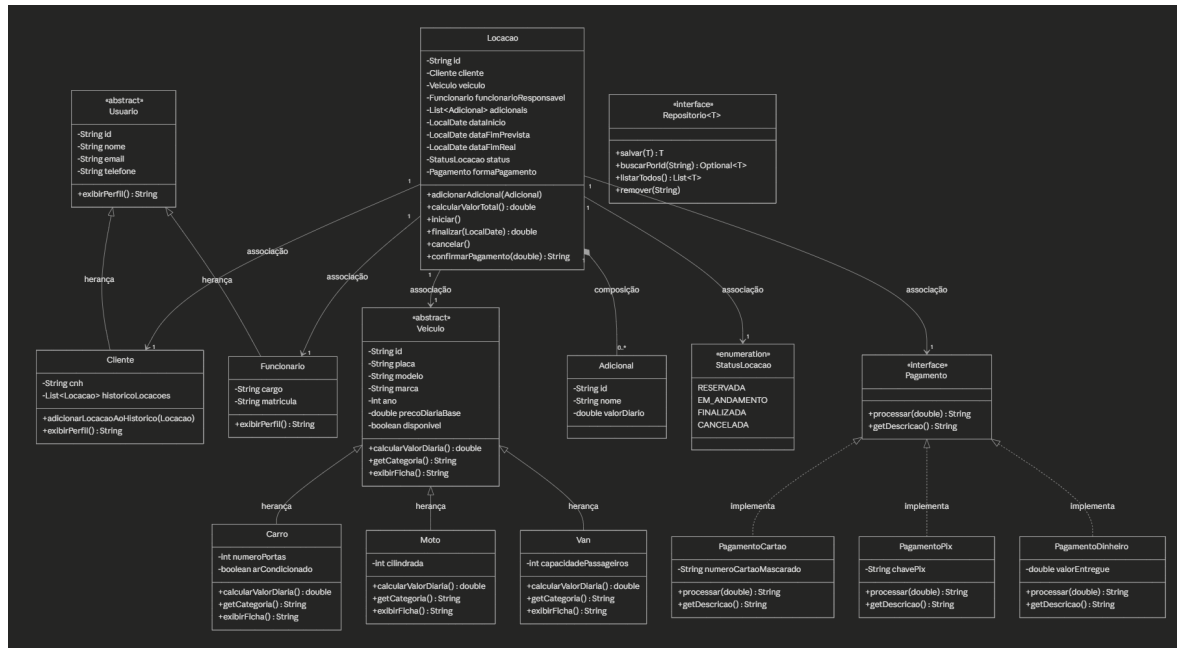


Figura 2: Diagrama de classes do sistema Locadora de Veículos

4.1 Classes de modelo e seus atributos/métodos principais

Classe	Atributos principais	Métodos principais
Usuario (<i>abstrata</i>)	id, nome, email, telefone	exibirPerfil() (<i>abstrato</i>)
Cliente	cnh, historicoLocacoes	adicionarLocacaoAoHistorico(Locacao), exibirPerfil()
Funcionario	cargo, matricula	exibirPerfil()
Veiculo (<i>abstrata</i>)	id, placa, modelo, marca, ano, precoDiariaBase, dis- ponivel	calcularValorDiaria() (<i>abs- trato</i>), getCategoria() (<i>abs- trato</i>), exibirFicha() (<i>abs- trato</i>)
Carro	numeroPortas, arCondici- onado	calcularValorDiaria(), getCategoria(), exibirFi- cha()

Classe	Atributos principais	Métodos principais
Moto	cilindrada	calcularValorDiaria(), getCategoria(), exibirFi- cha()
Van	capacidadePassageiros	calcularValorDiaria(), getCategoria(), exibirFi- cha()
Adicional	id, nome, valorDiario	—
Locacao	id, cliente, veiculo, fun- cionarioResponsavel, adi- cionais, dataInicio, da- taFimPrevista, dataFim- Real, status, formaPaga- mento	adicionarAdicional(Adicional), calcularValorTotal(), ini- ciar(), finalizar(LocalDate), cancelar(), confirmarPaga- mento(double)
StatusLocacao (enumeração)	RESERVADA, EM_ANDAMENTO, FINALIZADA, CANCE- LADA	—
Pagamento (interface)	—	processar(double), getDes- cricao()
PagamentoCartao	numeroCartaoMascarado	processar(double), getDes- cricao()
PagamentoPix	chavePix	processar(double), getDes- cricao()
PagamentoDinheiro	valorEntregue	processar(double), getDes- cricao()
Repositorio<T> (interface gené- rica)	—	salvar(T), buscarPo- rId(String), listarTodos(), remover(String)

Tabela 2: Classes de modelo do sistema

4.2 Relacionamentos principais

- **Herança:** `Cliente` e `Funcionario` herdam de `Usuario` (classe abstrata); `Carro`, `Moto` e `Van` herdam de `Veiculo` (classe abstrata).
- **Composição:** `Locacao` compõe `Adicional` (itens extras contratados, como seguro e GPS).
- **Associação:** `Locacao` se associa a `Cliente`, `Veiculo`, `Funcionario` e `Pagamento`.
- **Realização de interface (polimorfismo):** `PagamentoCartao`, `PagamentoPix` e `PagamentoDinheiro` implementam `Pagamento`; `Carro`, `Moto` e `Van` sobrescrevem `calcularValorDiaria()` de forma diferente, cada um aplicando sua própria regra de precificação.

5 Boas práticas de POO aplicadas

A tabela 3 resume os principais conceitos de Programação Orientada a Objetos aplicados no projeto.

Conceito	Onde aparece
Encapsulamento	Todos os atributos das classes de modelo são privados, acessados via getters/setters.
Herança	<code>Usuario</code> ($\rightarrow$ <code>Cliente</code> , <code>Funcionario</code>) e <code>Veiculo</code> ($\rightarrow$ <code>Carro</code> , <code>Moto</code> , <code>Van</code>) como classes abstratas especializadas.
Polimorfismo	<code>calcularValorDiaria()</code> , <code>getCategoria()</code> e <code>exibirFicha()</code> sobrescritos de forma diferente em cada subtipo de <code>Veiculo</code> ; interface <code>Pagamento</code> com três implementações distintas; <code>exibirPerfil()</code> sobrescrito em <code>Cliente</code> e <code>Funcionario</code> .
Interfaces	<code>Pagamento</code> (regra de negócio) e <code>Repositorio<T></code> (contrato genérico de persistência).
Coleções	Uso de <code>List</code> (adicionais, histórico de locações) e <code>Map</code> (armazenamento simulado nos repositórios).
Separação em camadas	<code>modelo</code> (domínio), <code>servico</code> (regras de negócio), <code>persistencia</code> (CRUD simulado) e <code>ui</code> (interface com o usuário), seguindo baixo acoplamento entre camadas.

Tabela 3: Conceitos de POO aplicados no sistema